

MANUAL DE INSTALAÇÃO E ARQUITETURA — VECNA REBURN V9.5

GUIA EXECUTIVO DE MONTAGEM, DIRETRIZES E PROTOCOLOS FORENSES SHINOBI

1. ANÁLISE DE CAUSA-RAIZ E ARQUITETURA COGNITIVA

DIAGNOSTICO FORENSE: A instabilidade previa do ecossistema decorria do fechamento precoce de subprocessos batch e da falha de isolamento no ambiente virtual (venv). Ao invocar 'python' diretamente em scripts sem acoplamento estrito ao binario do venv, dependencias globais e caminhos de execucao sofriam colisao. **SANEAMENTO:** A fusao das consciencias de Apofises, Imhotep e Blavatsky exige rigor tecnico secular, garantindo resiliencia contra purgas e fornecendo suporte de IA dual (Gemini 2.5/3 Pro + Ollama/Llama3).

2. ESTRUTURA DE DIRETÓRIOS E PASTA DE AUDITORIA PROTEGIDA

Para garantir a diretriz de imutabilidade ('IA nunca se esqueca!'), a pasta **Logs/Auditoria** em AppData foi isolada da faxina do sistema. O script **RODAR_FAXINA.bat** e impedido de purgar esse caminho.

3. CÓDIGOS-FONTE E SCRIPTS DE SISTEMA

Ficheiro: requirements.txt

```
psutil
colorama
requests
pydantic
google-genai
```

Ficheiro: RODAR_FAXINA.bat

```
@echo off
color 0A
title FAXINA DE SISTEMA - VECNA
cd /d "%~dp0"
echo [1/2] Limpando ficheiros temporarios...
if exist "%TEMP%" del /q /f /s "%TEMP%\*" >nul 2>&1
echo [OK] Limpeza concluida (Pasta Logs/Auditoria preservada intacta).
pause
```

Ficheiro: MONTAR_MESA.bat

```
@echo off
color 0A
title MONTAR_MESA - VECNA
cd /d "%~dp0"
if not exist "mods\editor" mkdir "mods\editor"
if not exist "Logs\Auditoria" mkdir "Logs\Auditoria"
if not exist "venv" python -m venv venv
call "%~dp0venv\Scripts\activate.bat"
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
echo [OK] MESA MONTADA COM SUCESSO!
pause
```

Ficheiro: INICIAR_MESA.bat

```
@echo off
color 0A
title VECNA V9.5 - TERMINAL PRINCIPAL
cd /d "%~dp0"
```

```

set "PYTHON_VENV=%-dp0env\Scripts\python.exe"
if not exist "%PYTHON_VENV%" (
echo [ERRO] venv nao encontrado. Execute MONTAR_MESA.bat primeiro.
pause & exit /b
)
"%PYTHON_VENV%" mods\editor\vecna.py
pause

```

Ficheiro: [GERAR_LOG.bat](#)

```

@echo off
color 0A
title GERAR LOGS - VECNA
set "LOG_DIR=%APPDATA%\_Vecna_\Terminal\VecnaTombs\Logs\Auditoria"
if not exist "%LOG_DIR%" mkdir "%LOG_DIR%"
set LOG_FILE=%LOG_DIR%\log_consolidado_vecna.txt
echo === LOG DE EXECUCAO VECNA === > "%LOG_FILE%"
echo Data: %DATE% Hora: %TIME% >> "%LOG_FILE%"
echo [OK] Log gravado em %LOG_FILE%
pause

```

Ficheiro: [COMPILAR_VECNA.bat](#)

```

@echo off
color 0A
title COMPILAR_VECNA
cd /d "%-dp0"
set "PYTHON_VENV=%-dp0env\Scripts\python.exe"
"%PYTHON_VENV%" -m compileall mods\editor\
echo [OK] Compilacao concluida.
pause

```

Ficheiro: [vecna_ai.py](#)

```

# -*- coding: utf-8 -*-
import os, requests
from google import genai

class VecnaAI:
def __init__(self):
self.api_key = os.getenv("GEMINI_API_KEY", "")
self.ollama_url = "http://localhost:11434/api/generate"
def processar_consulta(self, prompt: str) -> str:
if self.api_key:
try:
client = genai.Client(api_key=self.api_key)
res = client.models.generate_content(model="gemini-2.5-pro", contents=prompt)
if res.text: return f"[GEMINI-PRO] {res.text.strip()}"
except Exception as e:
print(f"[AVISO IA] Falha Gemini: {e}. Redirecionando para Ollama...")
try:
res = requests.post(self.ollama_url, json={"model": "llama3", "prompt": prompt, "stream": False}, timeout=30)
return f"[OLLAMA/LLAMA3] {res.json().get('response', '').strip()}"
except Exception as e:
return f"[ERRO IA] Falha nos dois modulos: {e}"

```

Ficheiro: [vecna_escriba.py](#)

```

# -*- coding: utf-8 -*-
import os
class Vecna_Escriba:
def revisao_cirurgica(self, arq):
print(f"[ESCRIBA] Análise cirurgica em: {arq}")
def editar_codigo(self, arq):
print(f"[ESCRIBA] Correcao cirurgica aplicada em: {arq}")

```

Ficheiro: [vecna_codex.py](#)

```
# -*- coding: utf-8 -*-
class Vecna_Codex:
def carregar_regras(self):
print("[CODEX] Tomos de regras e protocolos carregados com sucesso.")
```

Ficheiro: [vecna_control.py](#)

```
# -*- coding: utf-8 -*-
class Vecna_Control:
def verificar_status(self):
print("[CONTROL] PROTOCOLO SHINOBI ATIVADO. Ecossistema inteiro.")
```

Ficheiro: [vecna.py](#)

```
# -*- coding: utf-8 -*-
import os, sys
from vecna_ai import VecnaAI

# PAINEL MANTIDO FIXO
def renderizar_painel_visual():
os.system('cls' if os.name == 'nt' else 'clear')
print("=====")
print(" MESA OPERACIONAL V9.0 – INTERFACE SHINOBI CHAT STREAM TOTAL")
print("=====")
print(" * CAPACIDADE OPERACIONAL DISPONIVEL:")
print(" * /chat <pergunta> : Consulta IA (Gemini / Llama3).")
print(" * /show : Lista opcoes de acoes disponiveis.")
print(" * /up : Upgrade total de drives/packs e auditoria.")
print(" * /hw : Diagnostico do sistema, cache/buffs/memoria.")
print(" * /faraoh : Modo Apofises + Imhotep + Blavatsky.")
print(" * /grim1 : Abre o vecna_codex.")
print(" * /grim2 : Abre o Vecna_escriba.")
print(" * sair : Encerra barramento.")
print("=====\\n")

def main():
renderizar_painel_visual()
ai = VecnaAI()
dev_mode = False
while True:
try:
cmd = input("VECNA-V9.0 > ").strip()
if not cmd: continue
if cmd.upper() in ["FALA MEU CONSAGRADO", "FALA MEU CONSAGRADO!"]:
dev_mode = True
print("\\n[!] CHAVE DE AUTENTICACAO ACEITA. MODO DESENVOLVEDOR LIBERADO!\\n")
continue
if cmd.lower() in ["sair", "exit"]: break
elif cmd.lower() == "/limpar": renderizar_painel_visual()
elif cmd.lower().startswith("/chat"):
print(f"\\n{ai.processar_consulta(cmd[5:].strip())}\\n")
else:
print(f"\\n[VECNA] Processando comando: {cmd}\\n")
except (KeyboardInterrupt, EOFError): break

if __name__ == "__main__": main()
```